Documentation: config.json

Overall Purpose

Think of this file as the **settings panel** or the **brain** of your EDR agent.

Instead of writing settings directly into the Python code (which is inflexible), we use this JSON file. This allows you to change what the agent monitors, how it behaves, and where it sends alerts, all without ever touching the Python script itself. This is a very common and powerful practice in software development.

JSON stands for **J**ava**S**cript **O**bject **N**otation, but it's just a simple text format for storing and organizing data using key-value pairs (like a Python dictionary).

Section-by-Section Breakdown

1. agent

This section contains general settings for the EDR agent itself.

JSON

```
"agent": {
  "id": "ubuntu-juiceshop-vm-01",
  "log_level": "info",
  "heartbeat_interval_seconds": 60
```

},

- "id": A unique name for this agent. This is useful if you ever run multiple agents on different machines and need to know which one is sending an alert.
- "log_level": Controls how much detail the agent logs about its own operations. "info" is a good default. For debugging, you could change it to "debug" to get much more verbose output.
- "heartbeat_interval_seconds": A setting for a potential future feature where the agent "checks in" with a central server every 60 seconds to show it's still alive and running.

2. monitoring

This is the most important section. It tells the agent **exactly what to watch**. Each part can be turned on or off with "enabled": true/false.

JSON

```
"monitoring": {
  "file_integrity": { ... },
  "process_monitoring": { ... },
  "network_monitoring": { ... },
  "juice_shop_monitoring": { ... },
  "ssh_monitoring": { ... },
  "log_sources": [ ... ]
},
```

- "file_integrity": Watches important system files for any changes. An attacker might try to modify these files to gain more permissions or hide their tracks. We are watching:
  - /home/james/juice-shop: The application's own folder.
  - /etc/passwd, /etc/shadow: Files that store user account information.
  - /etc/sudoers: A file that controls who can run commands with administrator privileges.
- "process_monitoring": Keeps an eye on running programs (processes). Here, it's configured to make sure the main Juice Shop application process is running as expected.
- "network_monitoring": Checks which network ports are open and listening for connections on the machine. We're making sure port 3000, the default for Juice Shop, is open.

- "juice_shop_monitoring": This is a special, custom monitor just for Juice Shop. It watches the application's access logs for a flood of errors ("error_threshold": 10) in a short time window ("error_time_window_minutes": 1), which could signal a web attack.
- "ssh_monitoring": Watches the system's authentication log (/var/log/auth.log) for failed SSH login attempts. If it sees 5 failures within 5 minutes, it could mean someone is trying to guess a password (a brute-force attack).
- "log_sources": A general list of other log files the agent should read and analyze for suspicious activity.

3. detection_rules

This section gives the agent a list of "bad things" to look for within the files and logs it's monitoring.

JSON

```
"detection_rules": {
    "suspicious_commands": [ ... ],
    "web_attack_patterns": [ ... ]
},
```

- "suspicious_commands": A list of commands an attacker might run to figure out more about the system (e.g., whoami, cat /etc/passwd). The agent will look for these commands in the logs.
- "web_attack_patterns": A list of text patterns that often appear during web attacks. For example, SELECT.*FROM is a pattern for SQL Injection, and <script> is a pattern for Cross-Site Scripting (XSS).

4. output

This tells the agent where to save its own findings.

JSON

```
"output": {
  "type": "file",
  "filepath": "/var/log/edr_agent.log"
}
```

- Currently, it's set to write all its logs to a single file: /var/log/edr_agent.log.

Documentation: edr_agent.py

Overall Purpose and Architecture

This Python script is the **engine** of the EDR agent. It reads the config.json file and then starts up all the necessary monitoring tasks.

To understand this code, it's helpful to use an analogy:

- The **EDRAgent class** is like a **Project Manager**. It doesn't do the monitoring work itself.
- The individual **monitors** (for files, processes, etc., which are in other files) are like **Specialist Workers**.
- The Project Manager (EDRAgent) hires each Specialist Worker and gives them their own workspace to run in. This is called **multi-processing**. It means each monitor runs independently, so if one crashes, it won't bring down the whole agent.
- The workers communicate back to the manager using a shared **Queue**, which is like a digital message inbox. When a monitor detects something suspicious, it puts an alert message into the queue.
- The manager constantly checks this queue for new messages and decides what to do with them, like logging the alert or taking action.

Code Breakdown

1. Imports and Initial Setup

Python

```
#!/usr/bin/env python3
```

```
import time
import os
import logging
import json
# ... and so on

# Add the project root to the Python path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

# Import the base class that all monitors will inherit from
from monitors.base_monitor import BaseMonitor
```

- **Imports**: The script starts by importing all the Python libraries it needs.
  - multiprocessing: This is the key library that allows the agent to run each monitor in a separate process.
  - logging: Used for creating log messages.
  - json: Used to read and parse the config.json file.
  - pkgutil, importlib, inspect: These are used together to *dynamically* find and load the monitor modules from the monitors/ directory.
- **sys.path.insert(...)**: This is a helper line that tells Python, "When you're looking for files to import, also look in the main folder of our project." This makes it easy to import our own monitor files.

2. The JsonFormatter Class

Python

```
class JsonFormatter(logging.Formatter):
    """
    Formats log records as a single line of JSON.
    """
    def format(self, record):
        # ... code to convert a log message to JSON ...
        return json.dumps(log_object)
```

- **Purpose**: By default, Python's logging looks like INFO:EDRAgent:Agent is starting.... This is easy for humans to read but hard for other programs to analyze.

- **What it does**: This custom class intercepts every log message and converts it into a structured **JSON format**, like {"message": "Agent is starting...", "timestamp": "2025-09-04T13:01:34,123", "logger": "EDRAgent"}. This structured data is much easier to feed into dashboards, databases, or other security tools (like a SIEM).

3. The EDRAgent Class (The Project Manager)

This class defines the main logic for the agent.

__init__(self, config) (The Constructor)

- This method runs once when the agent is first created. It's the setup phase.
- self.config = config: It saves the configuration loaded from config.json.
- self.logger = self._setup_logging(): It sets up its own logger using our custom JsonFormatter.
- self.log_queue = Queue(): It creates the shared message inbox for all monitor processes to send alerts back to.
- self.shutdown_event = Event(): It creates a signal that can be used to tell all child processes to shut down gracefully.
- self.action_dispatcher = {...}: This is a dictionary that acts like a "switchboard" for active responses. It maps an action name (e.g., "kill_process") to the function that knows how to perform that action (self._handle_kill_process).

_load_and_start_monitors(self)

- This is one of the cleverest parts of the agent. Instead of having to manually import and start each monitor, this code **automatically discovers** them.
- It scans the monitors folder for Python files.
- For each file it finds, it imports the code and looks for a class that is a "child" of BaseMonitor. This ensures we only load actual, properly-coded monitors.
- For each valid monitor it finds, it creates a new **Process**, telling that process to run the monitor's code. It then starts the process and adds it to our list of running processes.

_process_log_queue(self) and _dispatch_response(self, alert)

- These methods are the core of the manager's work loop.
- _process_log_queue checks the log_queue for new messages from the monitors.
- For each message, it first logs the event.
- Then, it checks if the message contains an "action" key. If it does, it means a monitor is requesting the agent to do something actively (like block an IP).
- It then passes the alert to _dispatch_response, which uses the action_dispatcher switchboard to call the correct function (e.g., _handle_block_ip).

_handle_kill_process(self, alert) and _handle_block_ip(self, alert)

- These are the **active response** functions.
- _handle_kill_process extracts the Process ID (PID) from the alert and uses the psutil library to terminate it.
- _handle_block_ip extracts the IP address from the alert and runs a system command (sudo ufw ...) to add a firewall rule to block it.
- **Important Note**: As the code comment says, actions like this require the agent to be run with high privileges (e.g., as the root user or with specific sudo permissions). If it doesn't have these permissions, the actions will fail.

run(self)

- This is the main loop that keeps the agent alive.
- It starts by calling _load_and_start_monitors().
- It then enters an infinite while True: loop. Inside this loop, it constantly:
  1. Processes any new alerts from the log_queue.
  2. Checks to make sure all its monitor processes are still alive. If one has crashed, it logs an error.
  3. Sleeps for 1 second (time.sleep(1)) to prevent it from using 100% of the CPU.
- The try...except KeyboardInterrupt block ensures that if you press Ctrl+C in the terminal, the agent will shut down cleanly instead of just crashing.
- The finally block makes sure that, no matter what, it signals all monitor processes to stop and waits for them to finish before exiting.

4. The if __name__ == "__main__" Block

Python

```
if __name__ == "__main__":
    # ... code to load config ...
    agent = EDRAgent(config=CONFIG)
    agent.run()
```

- This is a standard Python construct. The code inside this block will **only** run when you execute the script directly from the command line (e.g., python3 edr_agent.py).
- It handles the startup sequence:
    1. Find and open config.json.
    2. Read the file and load the JSON data into a Python dictionary.
    3. Create an instance of our EDRAgent class, passing the configuration to it.
    4. Call the agent.run() method to kick everything off.